

Aula 02 – Dependências, Validação e CRUD em Memória

Objetivo da Aula: Entender como os pacotes Python funcionam por baixo dos panos, introduzir a biblioteca Pydantic para validação de dados de entrada e implementar os verbos HTTP restantes (POST, PUT, DELETE) para gerenciar o inventário.

1. A Descoberta do Pydantic e Atualização do Requirements

Antes de codificar, vamos desmistificar de onde vêm as ferramentas que usamos.

- **O Conceito:** Os pacotes de software raramente são construídos do zero. O FastAPI, por exemplo, não faz a validação de dados sozinho; ele "terceiriza" isso para uma biblioteca super-rápida chamada **Pydantic**.
- **A Prova no Terminal:** Abram o terminal (com o ambiente virtual venv ativado) e digitem: `pip list`

```
(venv) C:\Users\          \Desktop\api-inventario-maker>pip list
Package          Version
-----
annotated-doc     0.0.4
annotated-types   0.7.0
anyio             4.13.0
click            8.4.1
colorama          0.4.6
fastapi           0.136.3
h11              0.16.0
idna              3.16
pip              25.3
pydantic          2.13.4
pydantic_core     2.46.4
starlette        1.1.0
typing_extensions 4.15.0
typing-inspection 0.4.2
uvicorn          0.48.0
```

2. O Código Completo da Aula 02 (main.py)

Importar o Pydantic no `main.py` e substituir/complementar o código da aula anterior com a estrutura abaixo:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

# 1. Instanciação da API
app = FastAPI(title="API Inova Lab - Inventário Maker")

# 2. O Modelo Pydantic (A "Catraca" de Validação)
class ComponenteSchema(BaseModel):
    nome: str = Field(..., min_length=2, description="Nome do componente maker")
    quantidade: int = Field(..., ge=0, description="Quantidade em estoque (deve ser maior ou igual a zero)")
    categoria: str = Field(..., description="Categoria do item (ex: Atuadores, Microcontroladores)")

# 3. Nosso "Banco de Dados" temporário em memória
estoque_laboratorio = [
    {"id": 1, "nome": "Arduino Sensor Shield", "quantidade": 15, "categoria": "Placas de Expansão"},
    {"id": 2, "nome": "Micro Servo Motor SG90", "quantidade": 42, "categoria": "Atuadores"},
    {"id": 3, "nome": "Esteira em Acrílico", "quantidade": 2, "categoria": "Mecânica"}
]

# Rota Raiz
@app.get("/")
def raiz():
    return {"mensagem": "API do Laboratório Maker operante. Acesse /docs para ver a documentação."}

# CRUD - READ (Listar todos)
@app.get("/componentes")
def listar_componentes():
    return estoque_laboratorio

# CRUD - CREATE (Cadastrar novo item)
@app.post("/componentes", status_code=201)
def adicionar_componente(novo_componente: ComponenteSchema):
    # Lógica de autoincremento para o ID em memória
    if estoque_laboratorio:
        maior_id = max(item["id"] for item in estoque_laboratorio)
        novo_id = maior_id + 1
    else:
        novo_id = 1

```

```

# Converte o objeto Pydantic para dicionário e insere o ID
componente_dict = novo_componente.model_dump()
componente_dict["id"] = novo_id

estoque_laboratorio.append(componente_dict)
return {"mensagem": "Componente adicionado com sucesso!", "componente":
componente_dict}

# CRUD - UPDATE (Atualizar quantidade ou dados)
@app.put("/componentes/{componente_id}")
def atualizar_componente(componente_id: int, dados_atualizados: ComponenteSchema):
    for item in estoque_laboratorio:
        if item["id"] == componente_id:
            item["nome"] = dados_atualizados.nome
            item["quantidade"] = dados_atualizados.quantidade
            item["categoria"] = dados_atualizados.categoria
            return {"mensagem": "Componente atualizado com sucesso!", "componente":
item}

    raise HTTPException(status_code=404, detail="Componente não encontrado no
laboratório.")

# CRUD - DELETE (Remover item do inventário)
@app.delete("/componentes/{componente_id}")
def remover_componente(componente_id: int):
    for index, item in enumerate(estoque_laboratorio):
        if item["id"] == componente_id:
            estoque_laboratorio.pop(index)
            return {"mensagem": f"Componente com ID {componente_id} foi removido do
estoque."}

    raise HTTPException(status_code=404, detail="Componente não encontrado no
laboratório.")

```

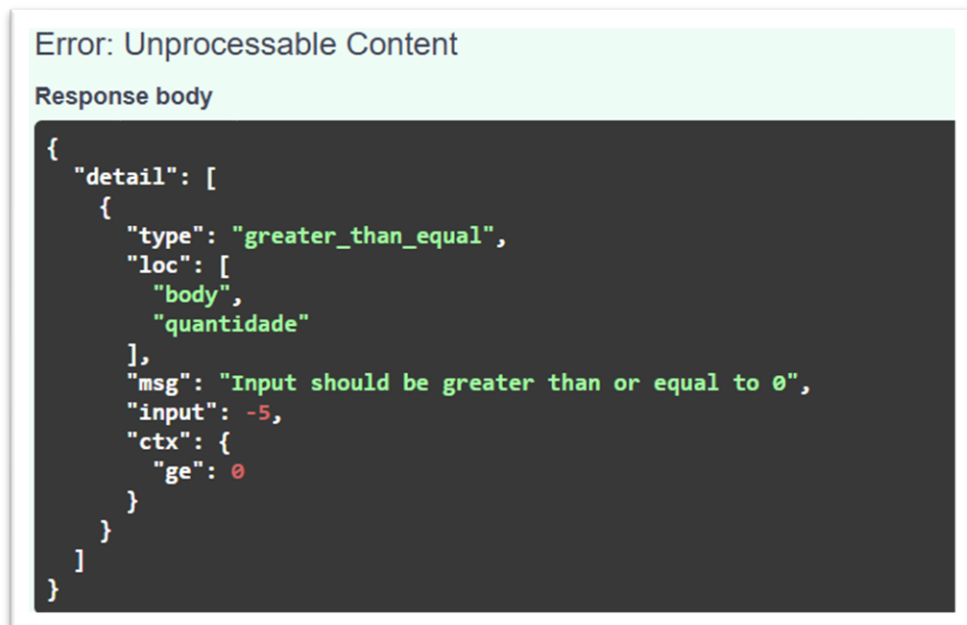
3. Pontos Chave

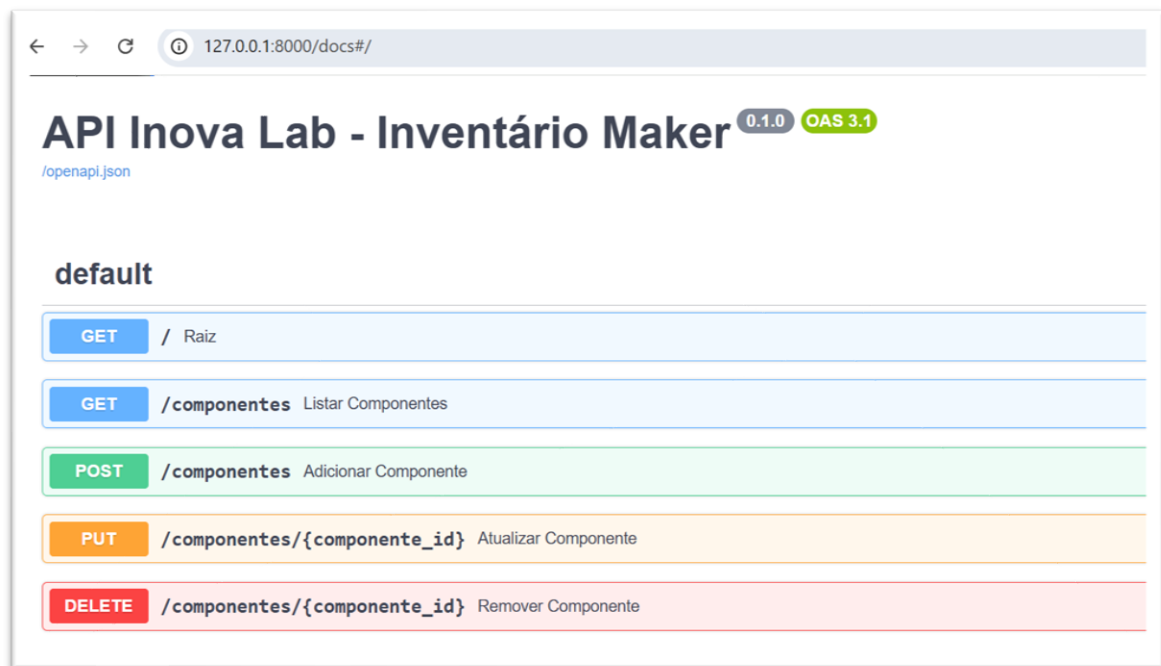
- **A Magia do Field no Pydantic:** O parâmetro *ge=0* (*greater than or equal to 0*). O Pydantic impede nativamente que o estoque fique negativo.
- **O Status Code de Criação (status_code=201):** Por padrão, o sucesso é 200 OK, mas quando *criamos* um recurso novo no servidor (como no método POST), o correto no padrão RESTful é retornar 201 Created.

- **Testando os Limites:** Abra o <http://127.0.0.1:8000/docs> e faça um **POST** com uma quantidade **-5**. O FastAPI, usando o Pydantic, barrará a requisição automaticamente com um erro 422 Unprocessable Entity, sem que tenha escrito sequer um "if" no código para isso.

Exemplo de corpo na requisição:

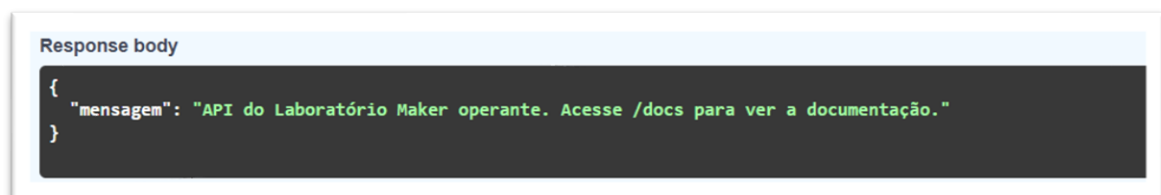
```
{  
  "nome": "Arduino",  
  "quantidade": -5,  
  "categoria": "Prototipação"  
}
```





Teste as rotas:

GET /



GET /componentes

Response body

```
[
  {
    "id": 1,
    "nome": "Arduino Sensor Shield",
    "quantidade": 15,
    "categoria": "Placas de Expansão"
  },
  {
    "id": 2,
    "nome": "Micro Servo Motor SG90",
    "quantidade": 42,
    "categoria": "Atuadores"
  },
  {
    "id": 3,
    "nome": "Esteira em Acrílico",
    "quantidade": 2,
    "categoria": "Mecânica"
  }
]
```

POST /componentes

Request body required

Edit Value | Schema

```
{
  "nome": "Arduino",
  "quantidade": 10,
  "categoria": "prototipação"
}
```

Response body

```
{
  "mensagem": "Componente adicionado com sucesso!",
  "componente": {
    "nome": "Arduino",
    "quantidade": 10,
    "categoria": "prototipação",
    "id": 4
  }
}
```

PUT /componentes/{componente_id}

Name	Description
componente_id * required integer (path)	4
Request body required	
Edit Value Schema <pre>{ "nome": "Ardu<u>ino</u> Novo", "quantidade": 10, "categoria": "<u>prototipação</u>"}</pre>	

Response body

```
{  "mensagem": "Componente atualizado com sucesso!",  "componente": {    "nome": "Arduino Novo",    "quantidade": 10,    "categoria": "prototipação",    "id": 4  }}
```

DELETE /componentes/{componente_id}

componente_id ★ required

integer

(path)

Execute

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/componentes/4' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/componentes/4
```

Server response

Code	Details
200	Response body <pre>{ "mensagem": "Componente com ID 4 foi removido do estoque." }</pre> Response headers

Como fechamento, você pode fazer um novo `git add .`, `git commit` e `git push` para salvar o progresso.

